ESTRUCTURAS DE DATOS Y ALGORITMOS · 31361

Grado en Diseño y Desarrollo de Videojuegos · Grado en Ingeniería Informática

Universidad San Jorge · Curso 2026-2027

PRÁCTICA INDIVIDUAL · CAMPAÑA DE 8 HORAS

Lord of EDA

Los cofres dispersos: listas dispersas en cuatro actos sobre las dos tablas de clase: sus operaciones, operaciones entre tablas, otras funciones hash y el juego de las colisiones

Tema	6. Tablas de dispersión
Modalidad	Individual
Dedicación	8 horas, en cuatro sesiones de 2 horas en clase (un acto por sesión)
Web	https://lordtables.vercel.app
Entrega	Una única entrega en la PDU, hasta el viernes 11 de diciembre de 2026 a las 23:59
Lenguaje	C, sobre el proyecto Listas_Dispersas de clase, el mismo en los cuatro actos

Objetivo de la campaña

Trabajar a fondo las tablas de dispersión del tema 6 en cuatro sesiones de 2 horas. Partes del proyecto de clase y, acto a acto, amplías sus dos tablas, la de listas y la del array: operaciones que las recorren y las modifican, operaciones entre dos tablas, otras funciones hash y otras formas de resolver las colisiones. Todo sobre **un solo proyecto base** y sobre **las mismas dos clases**, las cuatro sesiones.

Sesión	Se programa en	Ejercicios	Misiones
Acto 1 · El Gremio	TablaDispersaLista.c y TablaDispersaArray.c	1 a 5	1 a 5
Acto 2 · Las caravanas	TablaDispersaLista.c y TablaDispersaArray.c	6 a 9	6 a 9
Acto 3 · La sombra de la torre (funciones hash)	TablaDispersaLista.c y TablaDispersaArray.c	10 a 12	10 a 12
Acto 4 · El tesoro del dragón (colisiones)	TablaDispersaLista.c y TablaDispersaArray.c	13 a 15	13 a 16

La historia

> *Todo empezó con el Libro de los Cofres.*
>
> *Hace mucho, cuando los reinos aún no tenían nombre, los sabios del Gremio buscaron la forma de guardar todo lo que existía y encontrarlo al instante. No lo pusieron en fila, porque buscar en una fila es lento. Le dieron a cada cosa un número, y a cada número un hueco. Así nacieron las tablas dispersas: una de listas, con cubos donde caben muchos, y otra de casillas, para quien sabe buscar sitio cuando el suyo está ocupado.*
>
> *Durante siglos hubo paz. Pero el poder de encontrarlo todo despierta la codicia. Un rey muerto quiso llevarse su tesoro a la tumba. Un troll aprendió a cobrar por lo que no era suyo. Un nigromante descubrió que quien elige la función hash decide dónde cae cada cosa. Y en la montaña, un dragón empezó a amontonar lo que nadie podía contar. El Libro cayó en el olvido, las colisiones crecieron y lo que se perdía en la memoria se convertía en fantasma.*
>
> *Y el destino del Libro no recayó en un rey, ni en un mago, ni en un guerrero, sino en alguien que acababa de llegar al Gremio con un portátil y un proyecto de CLion. Tú.*

En el juego, cada objeto tiene su número, que es la clave, y se guarda cuántos hay, que es el elemento. La historia empieza en la aldea, la mañana de la fiesta de la cosecha. Merla, la maga que guarda el Libro, reúne una compañía para recuperarlo, acto a acto. Cada miembro te acompaña en una misión:



Merla, la maga archivera narra la campaña



Hilda, la herrera enana de las forjas



Bastián, el cartógrafo mediano del valle



Rina, la sanadora elfa del bosque



Ulf, el montaraz explorador de los pinares



Lis, la escudera va delante contra los jefes

Y cada acto termina con su jefe, que pone a prueba todo lo del acto:



Odric, el Rey de la Cripta acto 1



Grumak, el troll del puente acto 2



Vorhal, el nigromante de las cifras acto 3



Ascuaferro, el dragón acto 4, el jefe final

El proyecto base

Fichero	¿Se toca?	Qué es
`TablaDispersaLista.c`, `TablaDispersaArray.c`	Sí, lo que pone TODO	Las dos tablas de clase, con los ejercicios de los cuatro actos, cada acto en su bloque. Tienen dos campos más que en clase: la función hash que usa la tabla y, en el array, cómo busca sitio; por defecto son los de clase.
`alumno.h`	Sí	Tu nombre, que sale en los registros.
`main.c`	Si quieres	El de clase (target `pruebas`), para probar las tablas a mano.
`TablaDispersa*.h`	No	Las cabeceras, con lo que tiene que hacer cada función.
`main_mision.c`, `registro.c`, `registro.h`	No	Leen los eventos de cada misión, llaman a tus funciones, vigilan `malloc` y `free` y escriben el registro.
`eventos/`, `registros/`	No	Los eventos de las dieciséis misiones (si se cambian, la misión no cuenta) y los registros que escribe cada una.

Cómo se trabaja

1. Entra en la web de la práctica, **https://lordtables.vercel.app**, despliega las órdenes de Merla y descarga el proyecto base. Ábrelo en CLion con File > Open eligiendo la carpeta entera y escribe tu nombre en `alumno.h`. Es el mismo proyecto para los cuatro actos: guárdalo y sigue con él en cada sesión.
2. Haz los ejercicios en orden. Cuando acabes uno, elige en el desplegable de arriba a la derecha el target de su misión (`mision1`, `mision2`… hasta `mision16`) y ejecútalo: deja su registro en `registros/`.
3. En la web de la práctica (**https://lordtables.vercel.app**), elige el acto, entra en esa misión y sube allí su registro. La misión se juega sola, evento a evento: ves cómo quedan tus tablas, y el resultado de cada evento aparece al llegar a él. La nota de la misión, con sus estrellas, sale al final de la animación. Corrige, vuelve a ejecutar y vuelve a subir hasta que salgan todas las estrellas.
4. Cada acto termina con su jefe, la última misión, que vuelve a usar todo lo del acto. Todos los actos están abiertos en la web desde el principio.

> **Si tu programa se para a mitad** (un acceso a `NULL`, un índice fuera del array, un bucle que no termina), el registro llega hasta el evento donde se cortó y la web te dice en qué función mirar. Para encontrarlo, pon un punto de ruptura ahí y ejecuta con el depurador de CLion.

Qué comprueba la web en cada evento

- Que en cada tabla estén las claves y los valores que tocan, sin claves repetidas, y que `n` cuadre.
- En la de listas, que cada nodo esté en el cubo que le da la función hash de la tabla; en la del array, que cada clave se encuentre siguiendo su secuencia de búsqueda y, al insertar, que quede en la primera casilla libre de esa secuencia.
- Que las operaciones que mueven nodos (cambiar la clave, fusionar, poner al frente) muevan los mismos, que la copia no comparta ninguno y que las operaciones entre dos tablas no cambien la que solo miran.
- Lo que devuelve cada función.
- Que no haya fugas (bloques que siguen vivos y que ya no apunta nadie), dobles `free` ni punteros a memoria liberada.

Cada fallo dice de qué ejercicio es, y cada evento se juzga por lo que hace él solo, así que un fallo no arrastra a los siguientes. La web da una nota orientativa; la calificación la pone el profesor con tu código.

Entrega

- Una única entrega en la tarea de la práctica en la PDU, como muy tarde el **viernes 11 de diciembre de 2026 a las 23:59**.
- Se entrega un archivo ZIP con **la carpeta del proyecto entera** comprimida: el código, `alumno.h` y la carpeta `registros/` con los registros de todas las misiones que hayas hecho.
- Puedes subir el ZIP al acabar cada sesión y volver a subirlo después: cuenta el último que haya en la PDU al cerrar el plazo. Subirlo cada día es la mejor copia de seguridad.
- Nombre del archivo: **ListasDispersas_inicialApellido** (sin tildes ni espacios). Por ejemplo, `ListasDispersas_jperez.zip`.
- Los registros tienen que salir de ejecutar el código que entregas, con tu nombre en `alumno.h`.
- Si la carpeta `cmake-build-debug` hace el zip muy grande, puedes borrarla antes de comprimir: no hace falta.

Criterios de evaluación

Cada acto se califica sobre 10. La nota de la práctica es la media de los cuatro actos, y su peso en la calificación final es el que fija la guía docente de la asignatura.

Acto 1 · El Gremio

Criterio	Puntos	Qué se valora
Ejercicio 1 · el más abundante	1	Recorre toda la tabla, desempata por la clave y trata la tabla vacía.
Ejercicio 2 · medir las colisiones	1,5	La lista más larga y los cubos vacíos; el grupo más largo contando las BORRADA, y cuántas hay.
Ejercicio 3 · cambiar la clave	2	El mismo nodo a su cubo nuevo (también si es el primero), la suma si la nueva ya está y la BORRADA en el array.
Ejercicio 4 · quitar los agotados	2	Todos los que tocan, también el primero y los seguidos, con su `free` en la de listas y BORRADA en la del array.
Ejercicio 5 · juntar dos inventarios	2,5	Suma las claves repetidas, mueve los nodos sin `malloc` y deja el origen vacío.
Memoria y misiones	1	Sin fugas ni dobles `free`, y las cinco misiones superadas en los registros entregados.

Acto 2 · Las caravanas

Criterio	Puntos	Qué se valora
Ejercicio 6 · copiar	2,5	Libera lo que tenía destino y copia con nodos nuevos en el mismo orden; en el array, también las marcas.
Ejercicio 7 · comparar	1,5	Mismas claves y mismos elementos, sin depender del orden ni de las casillas.
Ejercicio 8 · lo que tienen los dos	2,5	Se queda con el menor, quita y libera lo que no está en los dos (también el primero de un cubo y los seguidos) y usa BORRADA.
Ejercicio 9 · pagar con lo del otro	2,5	Resta, quita lo que queda en 0 o menos con su `free` o BORRADA, y no toca la otra tabla.
Memoria y misiones	1	Sin fugas ni dobles `free`, y las cuatro misiones superadas en los registros entregados.

Acto 3 · La sombra de la torre

Criterio	Puntos	Qué se valora
Ejercicio 10 · plegamiento	3	Los trozos de dos cifras desde la derecha, la suma y el módulo, en las dos tablas.
Ejercicio 11 · mitad del cuadrado	3	El cuadrado sin desbordar (`long long`), las dos cifras del centro y el módulo, en las dos tablas.
Ejercicio 12 · polinómico	3	Las cifras en el orden pedido, `unsigned int` y el módulo, en las dos tablas.
Misiones	1	Las tres misiones superadas en los registros entregados.

Acto 4 · El tesoro del dragón

Criterio	Puntos	Qué se valora
Ejercicio 13 · cuadrática y doble	3	Las dos fórmulas con `indice` y el módulo, y `hash2`.
Ejercicio 14 · lo que cuesta buscar	2,5	Cuenta el nodo o la casilla final, no corta en las BORRADA, y el total suma solo lo que hay.
Ejercicio 15 · reorganizar	3,5	Limpia sin dejar BORRADA y con n = 0 antes de reinsertar; mueve el mismo nodo al frente, también cuando no hace falta moverlo.
Memoria y misiones	1	Sin fugas ni dobles `free`, y las cuatro misiones superadas en los registros entregados.

Acto 1 · El Gremio

Objetivo del acto

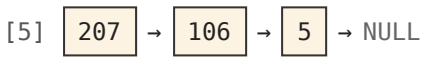
Ampliar las dos tablas de clase, la de **listas** (encadenamiento) y la del **array** (direccionamiento abierto con exploración lineal), con cinco operaciones: buscar el objeto con más unidades, medir las colisiones, cambiar la clave de un objeto, quitar los agotados y juntar dos inventarios.

El punto de partida

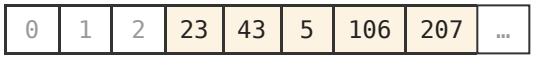
Las dos tablas de clase tal cual, con `TAM_LISTA = 101` cubos y `TAM = 20` casillas. Además ya viene hecha `actualizar`: si la clave ya está, cambia su elemento; si no, la inserta.

```
struct HashLista { Nodo *cubos[TAM_LISTA]; int n; };
struct HashArray { int claves[TAM]; int valores[TAM]; int marca[TAM]; int n; };
```

Listas (101 cubos): 5, 106 y 207 caen en el cubo 5



Array (20 casillas): 23 y 43 chocan en la 3



Lo que tienes que programar

Cinco ejercicios, cada uno en las dos tablas, al final de `TablaDispersaLista.c` y de `TablaDispersaArray.c`, en el bloque del acto 1, donde pone TODO.

Ejercicio 1 · El objeto con más unidades (misión 1)

`int masAbundante(h, int *clave, int *elemento)`: recorre toda la tabla (todos los nodos de todos los cubos, o todas las casillas OCUPADA) y deja en `*clave` y `*elemento` los del elemento más grande. Si hay empate, gana la clave más pequeña. Devuelve 1, o 0 si la tabla está vacía.

Ejercicio 2 · Medir las colisiones (misión 2)

`int cuboMasLargoHashLista(h, int *vacios)`: devuelve la longitud de la lista más larga y deja en `*vacios` cuántos cubos están a `NULL`. `int grupoMasLargoHashArray(h, int *borradas)`: devuelve el grupo más largo de casillas seguidas que no están VACIA (las BORRADA también cuentan), de la casilla 0 a la última y sin dar la vuelta, y deja en `*borradas` cuántas BORRADA hay.

Ejercicio 3 · Cambiar la clave de un objeto (misión 3)

`int cambiarClave(h, int vieja, int nueva)`: en la de listas se desengancha el **mismo nodo** (cuidado si es el primero), cambia su clave y se engancha al principio del cubo de la nueva, sin `malloc` ni `free`; si la nueva ya estaba, su elemento suma el de la vieja y el nodo de la vieja se libera. En la del array, la casilla de la vieja pasa a BORRADA y la nueva se suma o se inserta. Devuelve 0 si la vieja no está; si `vieja == nueva`, devuelve 1 sin hacer nada.

Ejercicio 4 · Quitar los agotados (misión 4)

`int eliminarSiMenor(h, int minimo)`: quita todos los objetos con un elemento menor que `minimo` y devuelve cuántos ha quitado. En la de listas, cada nodo se desengancha y se libera, aunque sea el primero de su cubo o haya varios seguidos. En la del array, cada casilla pasa a BORRADA, nunca a VACIA.

Ejercicio 5 · Juntar dos inventarios (misión 5)

`int fusionar(destino, origen)`: si la clave ya está en el destino, se suma; si no, se mete. En la de listas se **mueven los nodos** del origen, sin `malloc`, y si la clave ya estaba, el nodo del origen sobra y se libera. En la del array se reinserta cada OCUPADA del origen y devuelve 0 si alguna no cabe. El origen queda vacío.

Las misiones del acto 1

Target	Eventos	Qué prueba
`mision1` · La fiesta de la cosecha	20	Ejercicio 1, con Hilda: la mochila vacía, un solo objeto, empates que se desempatan por la clave más pequeña, claves negativas y un objeto que se adelanta.
`mision2` · El mapa de las colisiones	20	Ejercicio 2, con Bastián: las tablas vacías, un cubo que crece hasta cuatro nodos, una BORRADA en medio del grupo y claves que dan la vuelta al final del array.
`mision3` · El mercado de los trueques	21	Ejercicio 3, con Rina: el nodo del medio y el primero de un cubo, una clave que ya está, una que no existe, por sí misma y una negativa.
`mision4` · La cripta encantada	21	Ejercicio 4, con Ulf: el conjuro que quita los agotados (ninguno, el primero, varios seguidos, todos) y, después, buscar e insertar con las BORRADA.
`mision5` · Odric, el Rey de la Cripta	72	Ejercicio 5, el jefe, con Lis: dos inventarios, el suyo y el tesoro del rey muerto (las órdenes con `B`), que se fusionan una y otra vez.

Cómo repartir las 2 horas

Tiempo	Qué hacer
10 min	Abrir el proyecto, poner tu nombre, ejecutar `mision1` tal cual y subir su registro. Casi todo saldrá mal, y es lo esperado.
15 min	Ejercicio 1, con la misión 1.
20 min	Ejercicio 2, con la misión 2.
25 min	Ejercicio 3, con la misión 3.
25 min	Ejercicio 4, con la misión 4.
25 min	Ejercicio 5, con la misión 5, la del jefe.

Acto 2 · Las caravanas

Objetivo del acto

Trabajar con **dos tablas a la vez**: copiar una en otra sin compartir memoria, compararlas aunque guarden lo mismo en otro orden, quedarse con lo común y restar lo de una a la otra. Todo en las dos tablas de clase, la de listas y la del array. Al terminar deberías distinguir bien copiar un puntero de copiar un nodo, y saber cuándo hay que liberar.

El punto de partida

El mismo proyecto del acto 1, con las mismas dos tablas. Las misiones de este acto usan dos inventarios: el principal y el botín (las órdenes con `B`), que se crea la primera vez que se usa. Las órdenes nuevas siempre trabajan sobre los dos: el principal es el destino y el botín el otro.



Rina, la sanadora
misión 6



Ulf, el montaraz
misión 7



Bastián, el cartógrafo
misión 8



Lis, la escudera
misión 9



Grumak, el troll del puente
el jefe del acto

Lo que tienes que programar

Cuatro ejercicios, cada uno en las dos tablas, al final de `TablaDispersaLista.c` y de `TablaDispersaArray.c`, en el bloque del acto 2. Puedes usar las funciones de clase (`buscar`, `insertar`, `actualizar`…) dentro de las tuyas.

Ejercicio 6 · Copiar un inventario (misión 6)

`int copiar(destino, origen)`: destino queda igual que origen. En la de listas, primero se liberan los nodos que tuviera destino; después, cada cubo de origen se copia en el mismo cubo de destino, en el mismo orden, con **nodos nuevos** (`malloc`): la copia no puede compartir ni un nodo con el original. En la del array, las tres arrays casilla a casilla, marcas incluidas, y `n`. Origen no cambia. Devuelve cuántos elementos ha copiado.

Ejercicio 7 · Comparar dos inventarios (misión 7)

`int iguales(a, b)`: devuelve 1 si tienen las mismas claves con los mismos elementos, aunque dentro de un cubo estén en otro orden o en el array estén en otras casillas; 0 si no. No cambia nada.

Ejercicio 8 · Lo que tienen los dos (misión 8)

`int interseccion(destino, otro)`: destino se queda solo con las claves que también están en otro, y cada una con el **menor** de los dos elementos. Lo que se quita se libera en la de listas y pasa a

BORRADA en la del array. Otro no cambia. Devuelve cuántos elementos quedan en destino.

Ejercicio 9 · Pagar con lo que tiene el otro (misión 9)

`int diferencia(destino, otro)`: por cada clave de otro que esté en destino, al elemento de destino se le resta el de otro; si se queda en 0 o menos, se quita (`free` o BORRADA). Las claves de otro que no están en destino no hacen nada. Devuelve cuántos se han quitado.

Las misiones del acto 2

Target	Eventos	Qué prueba
`mision6` · El escriba de la caravana	22	Ejercicio 6, con Rina: copiar con colisiones; cambiar la copia y comprobar que el original no cambia; copiar encima de otra copia (sus nodos viejos sobran) y copiar un inventario vacío.
`mision7` · La balanza del mercader	20	Ejercicio 7, con Ulf: dos inventarios vacíos, lo mismo en otro orden y en otras casillas, un elemento distinto, un objeto de más y el mismo número de objetos pero distintos.
`mision8` · El pacto de las caravanas	30	Ejercicio 8, con Bastián: quedarse con el menor, quitar los dos primeros de un cubo, una clave que en el array queda detrás de las casillas quitadas y un pacto con una caravana vacía.
`mision9` · Grumak, el troll del puente	40	Ejercicio 9 y todo el acto, el jefe, con Lis: los peajes del troll que se quedan justos, que no llegan y que piden lo que no hay, y copias, comparaciones y pactos entre medias.

Cómo repartir las 2 horas

Tiempo	Qué hacer
30 min	Ejercicio 6, con la misión 6. La copia de la de listas es la más delicada: engancha cada nodo nuevo al final de su cubo.
20 min	Ejercicio 7, con la misión 7.
30 min	Ejercicio 8, con la misión 8.
40 min	Ejercicio 9 y la misión 9, el jefe, que vuelve a usar los cuatro.

Acto 3 · La sombra de la torre

Objetivo del acto

Ver que la función hash es una decisión de diseño: programar otras tres funciones del tema 6 (plegamiento, mitad del cuadrado y una polinómica sobre las cifras) y comparar cómo reparte cada una las mismas claves frente a la de clase. Al terminar deberías saber calcular a mano el cubo de una clave con cada función y reconocer qué claves hacen chocar a cada una.

El punto de partida

Las mismas dos tablas. Cada una guarda qué función hash usa (`funcion`, que por defecto es `HASH_CLASE`, la de siempre) y todas las operaciones la usan a través de `indice(h, clave)`. La orden `FUNCION` de las misiones la cambia llamando a `usarHashLista` y `usarHashArray`, que ya están hechas y solo funcionan con la tabla vacía. En cada misión tu inventario usa la función nueva y el botín la de clase (o otra), así que las mismas claves se ven repartidas de dos formas.

```
#define HASH_CLASE 0            clave mod TAM
#define HASH_PLEGAMIENTO 1      trozos de dos cifras sumados, mod TAM
#define HASH_MITAD_CUADRADO 2   cifras del centro del cuadrado, mod TAM
#define HASH_POLINOMICO 3       h = h*31 + cifra, mod TAM

struct HashLista { Nodo *cubos[TAM_LISTA]; int n; int funcion; };
struct HashArray {
    int claves[TAM]; int valores[TAM]; int marca[TAM]; int n;
    int funcion;
    int sondeo;     /* acto 4 */
};
```

Clave	Clase (mod 101)	Plegamiento	Mitad del cuadrado	Polinómico
1234	22	12 + 34 = 46	1522756 → 27	1
3412	79	34 + 12 = 46	11641744 → 17	70
2143	22	21 + 43 = 64	4592449 → 24	27



Bastián, el cartógrafo
misión 10



Hilda, la herrera
misión 11



Lis, la escudera
misión 12



Vorhal, el nigromante
el jefe del acto

Lo que tienes que programar

Tres funciones hash, cada una en las dos tablas (mod `TAM_LISTA` en `TablaDispersaLista.c` y mod `TAM` en `TablaDispersaArray.c`), en el bloque del acto 3. Las claves de este acto son positivas.

Ejercicio 10 · Plegamiento (misión 10)

`int hashPlegamiento(int clave)`: parte la clave en trozos de dos cifras empezando por la derecha (`clave % 100`, luego `clave / 100`…) y los suma: 12345 da 1 + 23 + 45 = 69. Devuelve la suma mod el tamaño.

Ejercicio 11 · Mitad del cuadrado (misión 11)

`int hashMitadCuadrado(int clave)`: eleva la clave al cuadrado en un `long long` (para que no se desborde) y se queda con las dos cifras del centro: quita las dos últimas y coge las dos siguientes, `(c / 100) % 100`. Devuelve eso mod el tamaño.

Ejercicio 12 · Polinómico sobre las cifras (misión 12)

`int hashPolinomico(int clave)`: `h = 0` y, cifra a cifra empezando por la última, `h = h * 31 + cifra`, con `h` `unsigned int`. Devuelve `h` mod el tamaño. Es la idea del hash de cadenas del tema, aplicada a las cifras.

Las misiones del acto 3

Target	Eventos	Qué prueba
`mision10` · El pergamino plegado	22	Ejercicio 10, con Bastián: claves con las mismas cifras cambiadas de sitio, que el plegamiento junta y la de clase separa (y al revés), una clave de tres trozos y cambiar de función con la tabla llena.
`mision11` · El espejo del cuadrado	26	Ejercicio 11, con Hilda: el cubo 5 de la de clase se reparte, pero los números redondos (100, 200, 300) caen todos en el mismo.
`mision12` · Vorhal, el nigromante de las cifras	31	Ejercicio 12 y todo el acto, el jefe, con Lis: su inventario con el polinómico y el botín de Vorhal con el plegamiento; cambiar la clave, agotar y fusionar con funciones distintas en cada tabla.

Cómo repartir las 2 horas

Tiempo	Qué hacer
35 min	Ejercicio 10, con la misión 10. Calcula a mano el cubo de un par de claves antes de programar.
35 min	Ejercicio 11, con la misión 11.
50 min	Ejercicio 12 y la misión 12, el jefe.

Acto 4 · El tesoro del dragón

Objetivo del acto

Jugar con las colisiones del direccionamiento abierto y medirlas: programar la **exploración cuadrática** y el **doble hashing** del tema 6 en el array de clase, contar cuánto cuesta buscar en las dos tablas y reorganizarlas para que cueste menos. Al terminar deberías saber explicar el agrupamiento, por qué con 20 casillas (que no es primo) la cuadrática no encuentra sitio aunque sobren casillas, y por qué las BORRADA alargan las búsquedas.

El punto de partida

Las mismas dos tablas. El array guarda cómo busca sitio (`sondeo`, que por defecto es `SONDEO_LINEAL`, la de clase), y `insertar`, `buscar`, `eliminar` y `actualizar` ya buscan llamando a `casillaHashArray(h, clave, intento)`, que tiene hecha la lineal. La orden `SONDEO` de las misiones lo cambia con `usarSondeoArray`, ya hecha, con la tabla vacía. Las claves de este acto son positivas.

```
lineal:      casilla(i) = (h + i)           mod 20      (ya hecha)
cuadratica:  casilla(i) = (h + i*i)         mod 20
doble:       casilla(i) = (h + i*h2(clave)) mod 20
h = indice(h, clave),   h2(clave) = 7 - clave mod 7
```

Las casillas que mira cada forma para la clave 23 (h = 3, h2 = 5), con el número de intento:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Lineal	·	·	·	0	1	2	3	4	5	·	·	·	·	·	·	·	·	·	·	·
Cuadrática	·	·	·	0	1	·	·	2	5	·	·	·	3	·	·	·	·	·	·	4
Doble (h2 = 5)	·	·	·	0	·	·	·	·	1	·	·	·	·	2	·	·	·	·	3	·



Bastián, el cartógrafo
misión 13



Hilda, la herrera
misión 14



Ulf, el montaraz
misión 15



Lis, la escudera
misión 16



Ascuaferro, el dragón
el jefe final

Lo que tienes que programar

En el bloque del acto 4 de las dos tablas. Empieza por el 13: las misiones usan la cuadrática y el doble.

Ejercicio 13 · Otras formas de buscar sitio (misión 13)

En `TablaDispersaArray.c`: `int hash2HashArray(int clave)`, el salto del doble hashing, `7 - clave mod 7`, y en `int casillaHashArray(h, int clave, int intento)` los casos que faltan: la cuadrática `(h + intento * intento) mod TAM` y la doble `(h + intento * hash2(clave)) mod TAM`, con `h = indice(h, clave)`.

Ejercicio 14 · Lo que cuesta buscar (misión 14)

`int costeBuscarHashLista(h, int clave)`: cuántos nodos de su cubo se miran hasta el de la clave (incluido), o todos si no está. `int costeBuscarHashArray(h, int clave)`: cuántas casillas se miran desde el intento 0 hasta la de la clave o la primera VACIA, las dos incluidas, como mucho `TAM`. Y `costeTotal` en las dos: la suma del coste de buscar cada clave que hay.

Ejercicio 15 · Reorganizar (misión 15)

`int limpiarBorradasHashArray(h)`: quita las BORRADA sin cambiar el tamaño: guarda las OCUPADA en dos arrays locales (de la casilla 0 a la última), pone todo VACIA y `n = 0` y las vuelve a insertar en ese orden con `insertarHashArray`. Devuelve cuántas BORRADA había. `int alFrenteHashLista(h, int clave)`: si la clave está, mueve **su nodo** al principio de su cubo, sin `malloc` ni `free`, para que la próxima vez se encuentre antes. Devuelve 1, o 0 si no está.

Las misiones del acto 4

Target	Eventos	Qué prueba
`mision13` · La escalera de la torre	24	Ejercicio 13, con Bastián: siete claves con la misma casilla, la cuadrática en su inventario y el doble en el botín; una que no cabe en la cuadrática aunque sobran 14 casillas, y las BORRADA.
`mision14` · La sala de los espejos	27	Ejercicio 14, con Hilda: el coste de buscar en las listas y en el array, lineal contra doble, claves que están, que no y después de borrar.
`mision15` · El gólem de polvo	26	Ejercicio 15, con Ulf: el gólem llena el array de BORRADA y el coste no baja hasta barrerlas; en las listas, lo que más se busca pasa delante.
`mision16` · Ascuaferro, el dragón de las colisiones	41	El jefe final, con Lis: plegamiento y cuadrática en su inventario, doble hashing en el tesoro del dragón, costes, barridos y, al final, fusionar todo.

Cómo repartir las 2 horas

Tiempo	Qué hacer
30 min	Ejercicio 13, con la misión 13. Comprueba a mano, con el dibujo de arriba, dónde tiene que caer cada clave.
25 min	Ejercicio 14, con la misión 14.
30 min	Ejercicio 15, con la misión 15.
35 min	La misión 16, el jefe final.